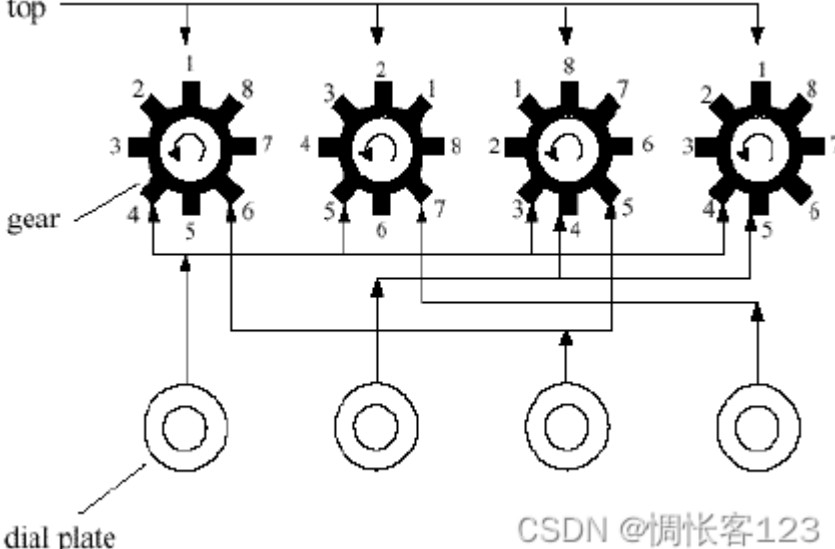


题目链接

本题是icpc亚洲区域赛北京赛区2004年的题目

题意

你的任务是帮助怪盗基德打开下图所示的一个密码锁。



该锁一共有 k ($1 \leq k \leq 20$) 个按钮和 k 个齿轮，每个齿轮上有 n ($2 \leq n \leq 10$) 个齿，分别印有整数 $1 \sim n$ ，其中最上面的那个齿上的数字在外部可见。初始时所有齿轮的可见数（即最上面齿的整数）均为1。每个按钮可以控制多个齿轮，用序列 $\{a_1, b_1, a_2, b_2, \dots, a_p, b_p\}$ 来描述，表示该按钮控制 p 个齿轮： $a_1, a_2, \dots, a_p$ ，其中每按一下这个按钮，齿轮 a_i 将逆时针旋转 b_i 个齿。当每个齿轮的可见数组成的序列恰好等于密码，锁将会打开。密码保证不是全1，因此初始情况下密码锁是锁上的。

无解时请输出No solution；有解时请输出最少的操作按钮的总次数。

分析

单模多元线性方程组求解，有解时最终不可避免地会枚举每一个解。即便用上剪枝技巧，数据如果刁钻的话，理论上是可以卡 $C^{20}(c \geq 3)$ 的时间，但如果真出现这种数据，只有tle了，因为有解时不可避免地会枚举每一个解。

线性方程组求解的一般思路是高斯消元，变成上三角阵回代（或者变成对角阵）求解。模线性方程组只能消元变成上三角阵回代求解。

注意点和可剪枝点：1、消元需要避免产生增根，并且在回代结束时进一步验证是否是增根；2、方程 $a_1x_1 + a_2x_2 + \dots + a_kx_k = b_i \pmod n$ 有解的条件是 $\gcd(n, a_1, a_2, \dots, a_k) \mid b_i$ ，用于剪枝；3、回代时可利用dfs实现剪枝（当前操作按钮的总次数不小于ans没必要继续）。

说一下消元避免增根的两个方法：1、辗转相除法；2、借助拓展欧几里德算法将基准方程的首系数 $a[i][i]$ 变成 $\gcd(a[i][i], a[j][i])$ ，再直接消元。

AC代码

这里给出借助拓展欧几里德算法消元的AC代码。

```
#include <iostream>
#include <numeric>
using namespace std;

#define K 22
int m[K][K], a[K][K], gm[K][K], g[K][K], ym[K][K], y[K][K], x[K],
    b, c, k, n, p, ans;

int gcd(int a, int b, int& x, int& y) {
    if (!b) {
        x = 1; y = 0; return a;
    } else {
        int g = gcd(b, a%b, y, x);
        y -= a/b*x;
        return g;
    }
}

void dfs(int p, int cnt = 0) {
    if (p < 0) {
        ans = min(ans, cnt);
        return;
    }
    for (x[p]=0; x[p]<n; ++x[p]) if (cnt+x[p] < ans && a[p][p]*x[p]%n == a[p][p]) {
        int i = -1;
        while (++i < p) {
            y[i][p] = a[i][k];
            a[i][k] = (a[i][k] - a[i][p]*x[p]%n + n) % n;
            if ((p==0 && a[i][k]) || (p && a[i][k] % g[i][p-1])) {
                break;
            }
        }
        if (i == p) {
            int j = -1;
            while (++j < k) {
                ym[j][p] = m[j][k];
                m[j][k] = (m[j][k] - m[j][p]*x[p]%n + n) % n;
                if ((p==0 && m[j][k]) || (p && m[j][k] % gm[j][p-1])) break;
            }
            if (j == k) --j, dfs(p-1, cnt + x[p]);
            while (j>=0) m[j][k] = ym[j][p], --j;
            --i;
        }
        while (i>=0) a[i][k] = y[i][p], --i;
    }
}

void solve() {
    for (int i=0; i<k; ++i) {
        cin >> a[i][k]; m[i][k] = a[i][k] = a[i][k]==1 ? 0 : n+1-a[i][k];
        for (int j=0; j<k; ++j) m[i][j] = a[i][j] = 0;
    }
    for (int i=0; i<k; ++i) {
        cin >> p;
        while (p-->0) cin >> c >> b, m[c-1][i] = a[c-1][i] = b;
    }
    for (int i=0; i<k; ++i) {
        gm[i][0] = gcd(n, m[i][0]);
        for (int j=1; j<k; ++j) gm[i][j] = gcd(gm[i][j-1], m[i][j]);
        if (a[i][k] % gm[i][k-1]) {
            cout << "No solution" << endl;
            return;
        }
    }
    for (int i=0; i<k; ++i) {
        for (int j=i; j<k; ++j) if (a[j][i]) {
            if (j > i) for (int x=i, t; x<=k; ++x) t = a[i][x], a[i][x] = a[j][x], a[j][x] = t;
            for (j=i+1; j<k; ++j) if (a[j][i]) {
                int x, y, g = gcd(a[i][i], a[j][i], x, y);
                if (x == 0) {
                    int z = a[i][i] / a[j][i];
                    for (x=i; x<=k; ++x) y = a[j][x], a[j][x] = (a[i][x] - z*a[j][x]) % n;
                } else {
                    for (int z=i; z<=k; ++z) a[i][z] = ((x*a[i][z] + y*a[j][z]) % n + n) % n;
                    for (x=i, y = a[j][i] / a[i][i]; x<=k; ++x) a[j][x] = (a[j][x] - y*a[i][x]) % n;
                }
            }
        }
        break;
    }
    g[i][i] = gcd(n, a[i][i]);
    for (int j=i+1; j<k; ++j) g[i][j] = gcd(g[i][j-1], a[i][j]);
    if (a[i][k] % g[i][k-1]) {
        cout << "No solution" << endl;
        return;
    }
}

ans = k*n;
dfs(k-1);
if (ans == k*n) {
    cout << "No solution" << endl;
    return;
}
cout << ans << endl;
}

int main() {
    while (cin>>k>>n && k) solve();
    return 0;
}
```